arXiv:2610.07457v1 [cs.LG] 5 Oct 2026

AlignQuant: TILE-ALIGNED MIXED-PRECISION QUANTIZATION FOR EFFICIENT LLM GENERATION

Hanzhi Zhang[1], Qiao Zhang[2], Qinglei Cao[2], Heng Fan[1], Yan Huang[1], Kewei Sha[3], Yunhe Feng[1]
[1]LLaVi Lab, Computer Science & Engineering, University of North Texas
[2]Computer Science, Saint Louis University; [3]Data Science, University of North Texas
{hanzhi.zhang,heng.fan,yan.huang,kewei.sha,yunhe.feng}@unt.edu
{qiao.zhang,qinglei.cao}@slu.edu

ABSTRACT

Fine-grained mixed-precision quantization promises efficient large language model inference, but local precision choices can conflict with regular GPU storage and computation units. This precision-boundary mismatch limits the translation of compression into practical acceleration. We introduce *AlignQuant*, a post-training quantization method that uses GPU-compatible two-dimensional weight tiles as the common unit of precision allocation, compact storage, and execution. This shared partition lets precision follow sensitivity within output channels. Joint prefill/decode calibration scores precision reductions using projection-output perturbations weighted by language-model loss gradients under quantized activations. Phase-normalized scores prioritize higher precision for tiles important to either phase under a model-wide weight-storage budget. Each tile stores one selected representation, while phase-specialized kernels reuse the packed model and expand lower-bit weights for INT8 computation with 8-bit activations. Across four LLMs spanning 3B to 14B parameters, *AlignQuant* achieves up to $2.50\times$ generation speedup over BF16 while preserving model quality. Evaluations further cover three GPUs and contexts up to 64K tokens. These results show that local precision flexibility and regular GPU execution can coexist through a shared tile unit. The implementation is available at https://github.com/HanzhiZhang-Ulrica/AlignQuant.

1 INTRODUCTION

Large language model (LLM) inference is increasingly constrained by parameter storage, memory traffic, and computation. Post-training quantization (PTQ) reduces these costs through lower-precision representations, while the resulting quantization error makes accuracy preservation a central challenge (Dettmers et al., 2022; Xiao et al., 2023; Frantar et al., 2022; Lin et al., 2024b). Uniform-bit quantization addresses this challenge by improving the quantized representation at a fixed weight precision (Frantar et al., 2022; Lin et al., 2024b). Mixed-precision quantization additionally exploits differences in sensitivity by assigning higher precision to sensitive weight regions under a weight-storage budget (Chen et al., 2024; Huang et al., 2024; Zheng et al., 2026; Li et al., 2026; Nie et al., 2026; Hooper et al., 2025). However, when fine-grained precision boundaries cross regular storage or GPU computation units, the resulting **precision-boundary mismatch** makes efficient deployment depend on how mixed-bit weights are packed, loaded, and converted for computation (Zhao et al., 2024; Lin et al., 2025).

Figure 1 illustrates how precision allocation, weight representation, and GPU execution jointly shape model quality and inference efficiency. Existing methods manage this tension through different precision-sharing and execution structures. Channel- and output-feature-level methods maintain regular partitions by sharing one bit-width across each channel or output feature, coupling weights with different sensitivities to the same precision choice (Chen et al., 2024; Zheng et al., 2026). Finer-grained approaches expose more local choices through quantization groups and hardware-aligned blocks, together with reordering, mixed formats, or specialized execution support (Huang et al., 2024; Li et al., 2026; Nie et al., 2026; Hooper et al., 2025). System-oriented quantizers co-design low-bit formats with weight layouts, dequantization, and GPU data paths to control these deployment costs (Zhao et al., 2024; Lin et al., 2025). These approaches expose a structural trade-off: coarser sharing restricts where higher precision can be placed, while finer allocation requires coor-

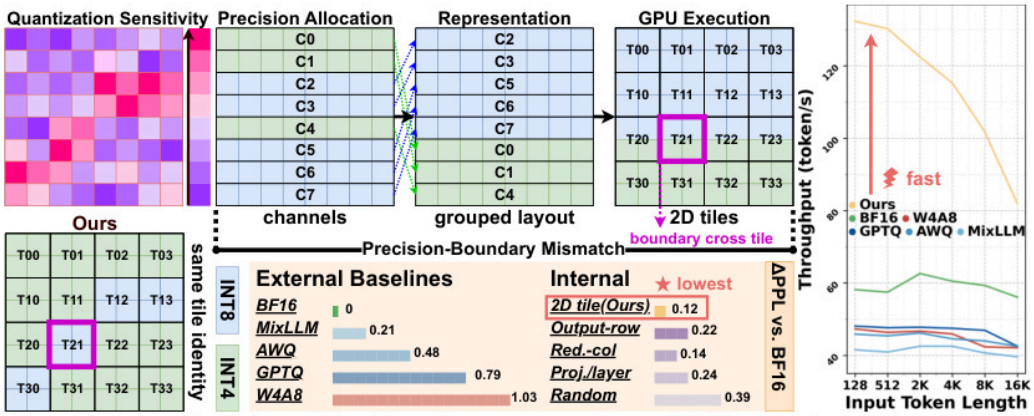

Figure 1: Comparison of precision allocation, representation, and GPU execution. Colors denote 8-bit/4-bit weight precision, and purple outlines track logical tiles. The schematic shows how channel-wise precision grouping can create a precision-boundary mismatch with regular two-dimensional GPU tiling, while *AlignQuant* uses a common weight-tile partition across allocation, storage, and execution. The quality panel reports WikiText-2 PPL increase over BF16, and the right panel reports generation throughput across prompt lengths on Llama-3.2-3B.

dinating the compact storage of heterogeneous weights with the formats used for GPU computation. This leads to our central question: *How can sensitivity-driven local precision choices be preserved in compact weight storage and direct GPU execution?*

Our key insight is to use GPU-compatible two-dimensional **weight tiles as a shared unit** for local precision selection, storage, and execution. Each tile receives its own bit-width, allowing precision to vary within output channels while providing a regular unit for storing and loading the selected weights. Our analysis supports this choice by showing that quantization sensitivity is localized within output channels. At a matched 4-bit tile fraction, two-dimensional tile allocation improves WikiText-2 perplexity over the output-row control. However, turning these local precision choices into efficient inference presents two coupled challenges. First, prefill processes prompt tokens and decode generates new tokens using the same weights. A shared precision assignment must therefore combine sensitivity evidence from both phases under one model-wide weight-storage budget. Second, the selected mixed-bit weights must retain their compact format in GPU memory and during loading while supporting integer computation tailored to each phase's workload (Lin et al., 2025).

We introduce *AlignQuant*, a tile-aligned mixed-precision method that constructs one compact model for efficient LLM generation. Joint loss-aware calibration assigns 4-bit or 8-bit precision under a model-wide budget, giving higher-precision priority to tiles important to either phase. Packing stores one selected representation per tile, preserving 4-bit weights in compact form through storage and loading. Phase-specialized kernels expand 4-bit weights on-chip to share INT8 computation with 8-bit weights and activations, directly consuming the same packed model.

Across Llama-3.2-3B, Qwen3-4B, Ministral-3-8B, and Qwen3-14B, *AlignQuant* achieves maximum $2.50\times$ inference speedups over BF16 on an NVIDIA A100, NVIDIA H100 NVL, and RTX 4090 GPUs while preserving model quality. These results show how preserving precision granularity through weight storage and computation translates local quantization choices into practical inference efficiency. Our contributions are as follows:

1. **Tile-Aligned Precision Design:** We develop a two-dimensional weight-tile organization that permits local bit-width choices within output channels. The same partition defines the regions evaluated during calibration and the units used for compact storage and GPU loading.
2. **Joint Prefill/Decode Allocation:** We develop a loss-aware rule that scores local projection-output changes from 8-to-4-bit substitutions under 8-bit activations. Joint prefill/decode calibration produces one precision map under a model-wide storage budget, prioritizing 8-bit retention for tiles important to either phase.
3. **Compact Storage and Direct Execution:** We develop prefill- and decode-specialized integer kernels that directly consume a compact model with one selected representation per tile. Compact 4-bit weight loads and on-chip expansion let both weight formats share INT8 arithmetic for efficient inference.

2 Related Work

Mixed-precision LLM quantization couples three design dimensions: the granularity at which precision varies, the representation used to realize heterogeneous precision, and the numerical transformations used to reduce quantization error.

Sensitivity-aware precision allocation has progressively moved from coarse model structure toward localized decisions. Layer-wise methods reduce the number of precision decisions and preserve regular operators, while coupling regions of different sensitivity to one bit-width across each layer (Lee et al., 2025; Zhao et al., 2025). Channel-, output-feature-, and group-wise methods capture finer heterogeneity through structured units, coupling the weights within each unit to one precision and, under layer-local budgets, constraining high-precision placement (Chen et al., 2024; Huang et al., 2024; Zheng et al., 2026; Harma et al., 2026). Block-level schemes expose more local choices through hardware-aligned partitions, using global budgets, Fisher-weighted perturbation, or outlier-aware clustering for bit-width selection (Li et al., 2026; Nie et al., 2026; Hooper et al., 2025; Xie et al., 2025). Per-weight salience aggregation captures additive contributions, while whole-block precision changes interact through the projection output (Nie et al., 2026; Hooper et al., 2025). Phase-adaptive methods vary precision across inference phases and generation steps, adding runtime selection among quantized weight representations (Chen et al., 2025). Allocation therefore depends on spatial granularity, sensitivity evidence, and precision sharing across inference phases.

Low-bit representation and execution approach the same trade-off from the systems side. Low-bit systems co-design numerical formats, packing, data movement, and kernels for efficient GPU computation (Dettmers et al., 2022; Zhao et al., 2024; Lin et al., 2025; Hu et al., 2025a). Within this category, uniform weight formats couple sensitive and less sensitive regions to the same bit-width (Lin et al., 2025; Hu et al., 2025a). Executable mixed-precision systems organize heterogeneous weights through output-feature partitions, reordered blocks, expert groups, or hardware-supported formats, tying their precision choices to corresponding layouts, indexing, or arithmetic paths (Zheng et al., 2026; Nie et al., 2026; Duanmu et al., 2025; Liu et al., 2026; Jang & Tambe, 2025). Representation-centric methods use sparse outliers, residual subspaces, reusable binary codes, or fractional-bit quantizers to improve fidelity or precision flexibility, with corresponding indexed accesses, auxiliary products, or specialized reconstruction (Dettmers et al., 2023; Saxena et al., 2024; Park et al., 2026; Lee & Song, 2026). Inference efficiency therefore depends on how compact representations map to actual GPU arithmetic.

Transformation-based quantization reshapes weight and activation distributions to reduce quantization error. Scaling methods protect salient weights or redistribute quantization difficulty between weights and activations, with flexibility limited to feature-wise rescaling (Xiao et al., 2023; Lin et al., 2024b). Rotation- and permutation-based methods redistribute outliers across feature dimensions, while transforms spanning precision partitions change the weights assigned to each quantized region (Ashkboos et al., 2024; Liu et al., 2025; Lin et al., 2024a). Affine and jointly optimized scaling and orthogonal transformations offer richer distribution shaping, while spatial bit-width placement remains a separate decision (Ma et al., 2024; Hu et al., 2025b; Sun et al., 2024). Runtime smoothing adapts channel scales during inference to control dynamic ranges, while loss-sensitive weight placement requires a separate allocation criterion (Yi et al., 2025). These approaches improve numerical fidelity, while executable mixed precision also requires compatible transformation, allocation, and execution partitions.

3 *AlignQuant*

AlignQuant uses a common two-dimensional weight-tile partition to connect **precision allocation, compact storage, and GPU execution**. Each tile uses 4-bit or 8-bit weights, while uniform 8-bit activations provide a common input format for hardware-accelerated INT8 computation. Offline calibration produces a precision map under a model-wide weight-storage budget. This map records the selected bit-width at each tile coordinate and determines the weights stored in one packed model shared by prefill and decode.

Figure 2 follows a tile from candidate construction and joint calibration to compact storage and integer computation. We evaluate each replacement of 8-bit weights with 4-bit weights through its local projection-output change, weighted by gradients of the full model loss. Prefill and decode provide separate calibration evidence, which is combined to select one representation per tile. The runtime uses the selected weights and scales with execution schedules adapted to the two phases.

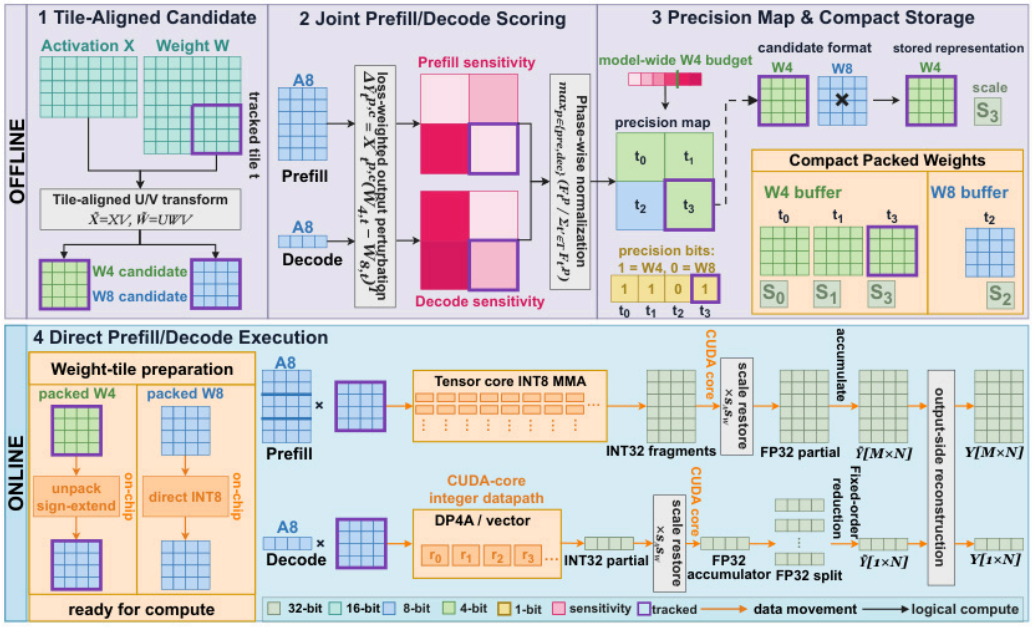

Figure 2: **Overview of *AlignQuant*.** Purple outlines track one logical weight tile. *(1) Candidates.* Tile-aligned U/V transforms prepare 4-bit and 8-bit weight candidates. *(2) Joint scoring.* Loss-weighted output changes score each 8-bit to 4-bit replacement in prefill and decode; the larger normalized score determines 8-bit priority. *(3) Compact storage.* The budget assigns 4-bit precision to the lowest-scoring tiles, storing one selected representation and scale per tile. *(4) Direct execution.* With 8-bit activations, prefill uses Tensor Core multiplication and decode uses DP4A dot products. The illustrated path expands 4-bit weights to INT8 on chip; long prefill reuses a temporary INT8 buffer. Scaled accumulation and output reconstruction produce Y.

3.1 Tile-Level Precision Assignment

We introduce offline precision allocation to select 8-to-4-bit replacements with the **smallest estimated impact under a storage budget**. The allocation leverages per-tile 4-bit and 8-bit candidates, multi-phase activation and gradient evidence, and a target 4-bit fraction.

For a linear projection $Y = XW^\top$, let $X \in \mathbb{R}^{M\times K}$ be the input activations and $W \in \mathbb{R}^{N\times K}$ the weights, where M is the number of activation rows and K, N are the input and output dimensions. Each row of W corresponds to an output feature and each column to an input feature. We partition all quantized weight matrices into equal-sized two-dimensional tiles, with model-wide set $\mathcal{T}$. Each $t \in \mathcal{T}$ is identified by its layer, projection, and matrix position, allowing **different regions within an output channel to use different bit-widths**.

Let $\rho \in [0, 1]$ denote the target fraction of 4-bit tiles. Since all tiles contain the same number of weights, each 8-to-4-bit replacement saves the same storage. Thus, the budget **determines how many tiles use 4-bit weights, while the scores determine which ones**.

Quantized candidates. As shown in Figure 2(1), candidate construction applies orthogonal input and output transformations (V along K, U along N) to redistribute outliers and minimize quantization error (Ashkboos et al., 2024; Liu et al., 2025), yielding symmetric 4-bit and 8-bit candidates per tile. Aligning groups with tile boundaries keeps each transformed weight tile ($\widetilde{W} = UWV$, $\widetilde{X} = XV$) within its assigned precision region. The paired output reconstruction preserves the floating-point projection (Appendix B.1).

Let $\widehat{W}_{4,t}$ and $\widehat{W}_{8,t}$ denote the reconstructed 4-bit and 8-bit candidates of the same transformed tile. Multiplying the values by their own scale puts both candidates in **the same numerical coordinates for comparison**. The precision assignment selects the integer values and scale retained for inference. Calibration and inference use the same 8-bit activation quantizer, so candidate comparisons reflect deployed inputs (Appendices B.2 and F.1).